Nine-Point vs. Four-Point Sampling in `mandelHybrid`'s Subdivision Heuristic

Matias Saavedra

Sunday 31st May, 2026

Binary: branch examine/9-points-sampling (intended tag binary-v3-9point) built on top of binary-v2-viz (616c147). Baseline for the comparison is binary-v2-viz itself — the 4-corner sampler — built from the same tree.

Resumen

Report 08-region-metrics-viz.tex pinned the binding load-balancing outliers to *minibrot interiors*: 960 × 540 regions whose four corners all reach `maxIter` (spread 0), so the 4-corner uniformity rule never subdivides them. This report implements the next-step 2b remedy — a **9-point stencil** that augments the four corners with the four edge midpoints and the centre — and measures it against the 4-corner baseline on 100 frames at 1920 × 1080 (i7-9750H + GTX 1660 Ti Max-Q, 12 threads, `diffT` = 0.1, `save` = 0). The headline is a *partial* win: the 9-point stencil exposes **+5.4 % more leaves** (5,323 → 5,608) and **eliminates the frame-89 outlier** (84.4 % interior), cutting the count of >5 s regions from **2 to 1**. But the densest region — frame 73, **97.3 % interior** — survives unchanged at 13.3 s, because all nine stencil points still land at `maxIter`. Crucially, end-to-end wall time is **statistically unchanged**: 52.39 s vs. 52.07 s over 3 reps each (+0.6%, inside the ∼1 % run-to-run spread). A Nsight Systems pass explains why, and corrects an earlier guess: the extra sampling costs a *measured* +0.40 s of CPU (+29 µs per `examine()`, ∼37 ms wall-equivalent) — negligible, not an "offset" — and splitting **conserves total compute** (≈595 s in both binaries). The heuristic moves work between threads but removes none; with the load already balanced under 2 %, the wall is bound by total interior-pixel work. The next optimisation must therefore *reduce* that work (periodicity checking or an interior certificate), not subdivide it further.

1. Setup and the Change Measured

Workload and machine.

Identical to report 08: the same `spec.in` (100 frames, 1920 × 1080, zoom ending at `maxIterations` = 10,000), the same i7-9750H (6c/12t) + GTX 1660 Ti Max-Q, 12 worker threads (1 GPU + 11 CPU), `diffThreshold` = 0.1, `pixelSizeThresh` = 32,768, `save` = 0 for pure-compute timing.

The change.

The 4-corner uniformity test in `MandelRegion::examine()` is extended to a 3×3 stencil: the four corners (as before) plus the four edge midpoints and the centre. The minimum and maximum iteration counts — and therefore the `(maxIter - minIter) < diffThresh * maxIter` decision — are taken over all nine points.

Código 1: mandelregion.cpp — the five points added to the uniformity test.

```
double midX = (upperX + lowerX) * 0.5;
double midY = (upperY + lowerY) * 0.5;
int extra[5] = {
  diverge (midX,   upperY),   // top edge midpoint
  diverge (midX,   lowerY),   // bottom edge midpoint
  diverge (upperX, midY),     // left edge midpoint
  diverge (lowerX, midY),     // right edge midpoint
  diverge (midX,   midY)      // centre
};
for (int i = 0; i < 5; i++) {
  if (extra[i] < minIter) minIter = extra[i];
  if (extra[i] > maxIter) maxIter = extra[i];
}
```

A child still inherits only its one shared outer corner (unchanged), so the five interior points are always recomputed: each `examine()` adds five `diverge()` calls over the 4-corner version. The split geometry and all other logic are untouched.

Baseline and measurement.

The 4-corner baseline is `binary-v2-viz` built in a separate worktree. Four measurement passes were taken on *both* binaries: (i) a clean wall-time A/B, three reps each, alternating to balance thermal drift; (ii) a single `quiet` = 0 metrics run for leaf counts, per-thread totals and the `[OUTLIER]` dump; (iii) a Nsight Systems capture (`nsys profile –trace=cuda,nvtx`) for the NVTX range breakdown, the CUDA API summary and the CUPTI kernel/mem-op statistics; and (iv) a `quiet` = 0 run whose per-region stderr lines give the full compute-time distribution. The `osrt` trace was omitted (OS-thread scheduling was not under analysis), keeping the capture lean.

2. Results

2.1. Work Decomposition

Tabla 1: Decomposition metrics, 100-frame hybrid run, `diffT` = 0.1. The GPU/CPU leaf split is scheduling-dependent; the total is deterministic.

Metric	4-point	9-point	Δ
Total leaf regions	5,323	5,608	+285 (+5.4%)
`examine()` calls	7,064	7,444	+380 (+5.4%)
Four-way splits	1,741	1,836	+95
GPU-computed leaves	4,053	4,189	+136
CPU-computed leaves	1,270	1,419	+149
GPU avg per region (ms)	11.56	11.01	−0.55
Outliers >5 s	2	1	−1

2.2. The Outliers

Tabla 2: CPU regions exceeding 5 s. The 9-point stencil removes the frame-89 region entirely; the denser frame-73 region survives.

Sampler	Frame	Size	Inset %	cardioid/bulb	CPU time
4-point	73	960×540	97.3%	no	14.5 s
4-point	89	960×540	84.4%	no	15.4 s
9-point	73	960×540	97.3%	no	13.3 s
9-point	89	— subdivided (no longer a leaf) —			

2.3. Load Balance and Wall Time

Tabla 3: Per-thread CPU compute totals and end-to-end wall time. A/B wall is the mean of 3 alternating reps; the parenthesised range is min–max.

Metric	4-point	9-point	Δ
CPU thread total (min–max, s)	51.0–51.9	50.4–51.1	tighter, lower
Sum CPU compute (s)	565.7	557.8	−7.9
Worst single region (s)	15.4 (f89)	13.3 (f73)	−2.1
Wall, A/B mean (s)	**52.07**	**52.39**	+0.32 (+0.6%)
A/B range (s)	51.84–52.34	52.10–52.80	overlapping

2.4. NVTX Range Breakdown and the Measured Sampling Overhead

The Nsight Systems capture resolves the cost of the five extra samples directly. Table 4 sums each NVTX range across all threads. Because the `examine region` range wraps the nested `compute` range, the pure subdivision overhead (corner + stencil sampling, the decision, and split bookkeeping) is `examine` − `CPU compute` − `GPU compute`.

Tabla 4: NVTX Push/Pop range summary (summed across all threads, Nsight Systems capture). The last row is the derived non-compute overhead; its Δ is the true cost of the 9-point stencil.

Range / quantity	4-point	9-point	Δ
`examine region` total (s)	598.34	603.84	+5.49
instances	7,064	7,444	+380
median (ms)	1.79	1.72	−0.07
max (s)	14.12	13.74	−0.38
`CPU region compute` total (s)	549.12	553.98	+4.86
`GPU region compute` total (s)	45.74	45.96	+0.22
Total compute (CPU+GPU, s)	594.86	599.94	+5.08 (+0.9%)
Subdivision overhead (s)	**3.49**	**3.89**	+0.40
per `examine()` (µs)	494	523	+29

The overhead delta is **+0.40 s of CPU work summed across eleven threads** — about 37 ms of wall-equivalent — or +29 µs per `examine()` call. Total compute (the pixel loops) is unchanged within noise (+0.9%): the same pixels are iterated regardless of how they are partitioned. The CPU-vs-GPU *split* of that total is scheduling-dependent and drifts a few percent run-to-run (this capture is a different run from Table 3), so only the aggregate total and the deterministic counts (leaves, `examine()` calls) should be read closely.

2.5. CUDA API and GPU Kernel (Nsight Systems)

Tabla 5: CUDA API time and CUPTI GPU statistics. `cudaMemcpy` time is ≈99 % kernel-wait (it equals the kernel total; the actual D→H transfer is the bottom row), unchanged by the stencil — the report-04 finding still holds.

Quantity	**4-point**	**9-point**
`cudaMemcpy` total / % of API	45.62 s / 99.4%	45.82 s / 99.3%
`mandelKernel` total (s)	45.47	45.56
avg / med / max (ms)	11.63 / 3.40 / 137.7	10.78 / 3.02 / 137.0
instances	3,910	4,228
D→H memcpy avg / max (µs)	23.2 / 79.7	22.6 / 82.1
D→H memcpy total (ms)	90.7	95.3

2.6. Per-Region Compute-Time Distribution

Tabla 6: Per-region compute time (ms) from the `quiet` = 0 run. The 9-point CPU distribution shifts down across the board — the extra splits make CPU regions smaller and remove the frame-89 tail. The GPU distribution is essentially unchanged.

Percentile	**4-pt CPU**	**9-pt CPU**	**4-pt GPU**	**9-pt GPU**
count	1,251	1,399	4,072	4,209
median	126.3	99.5	2.97	3.25
p90	1,003	987	28.1	27.8
p99	4,247	3,833	112	109
max	15,536	13,426	139	139

2.7. Visualization

The `viz=3` split-by-split animation for the 9-point binary is in `experiments/09-9point-sampling/viz_process/`; the 4-point counterpart is the report-08 animation in `experiments/08-region-metrics-viz/viz_process/`. Figure 1 contrasts the two on frame 89.

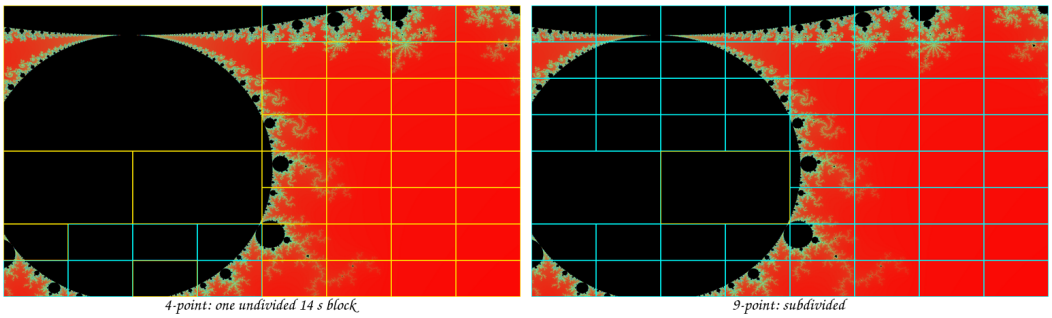

Figura 1: Frame 89 partition: 4-point (left) vs. 9-point (right). The 4-corner sampler leaves the upper-left minibrot body as coarse undivided cells — one of which is the 15 s outlier — because its four corners all reach `maxIter`. The 9-point stencil's interior samples detect spread and subdivide that body (more grid lines through the black region), removing the outlier. Cyan = GPU leaf, yellow = CPU leaf, grey = split skeleton.

3. Analysis

3.1. The stencil catches the 84%-interior outlier but not the 97% one

This is the headline. Both outliers have four corners pinned at `maxIter` (spread 0 under the 4-corner rule). The 9-point stencil adds five interior samples; whether it splits the region depends on whether *any* of those five lands on a pixel that diverges before `maxIter`.

For frame 89 (84.4 % interior) at least one interior sample fell on the ∼15.6 % of the region that is outside the minibrot, producing a non-zero spread and forcing the split — the region disappears from the outlier list. For frame 73 (97.3 % interior) all nine points landed on the ∼97 % that is inside, so the spread stayed 0 and the region computed as a single 13.3 s block. The mechanism is sparse sampling: the stencil catches a filament only when the filament crosses one of nine fixed points, and the densest interior regions have so little divergent area that thin filaments slip between the samples. The implication is that 9-point is a probabilistic improvement, not a guarantee — its hit rate falls as interior density rises.

3.2. More leaves, smaller GPU regions

The stencil exposed +5.4% leaves (Table 1) by detecting interior spread that the corners missed at many regions, not just the outliers. The GPU absorbed most of the new fine-grained work (its average region fell 11.56 → 11.01 ms, and it claimed +136 leaves), consistent with the report-04 observation that the GPU drains small regions fastest. The +95 extra splits are the new internal nodes feeding that growth.

3.3. Why wall time does not move — and it is not the sampling cost

Despite removing a 15 s outlier and shifting the whole CPU distribution down (Table 6: median 126 → 99 ms, p99 4247 → 3833 ms, max 15536 → 13426 ms), the A/B wall time is flat: +0.32 s (+0.6%), with the reps' ranges overlapping (9-point's fastest rep, 52.10 s, beats 4-point's slowest, 52.34 s). The Nsight Systems capture lets us say precisely why, and correct an earlier guess.

The sampling overhead is negligible — it does *not* offset the gain.

An earlier draft of this report attributed the flat result partly to the cost of the five extra samples. The NVTX breakdown (Table 4) refutes that: the non-compute overhead rises only from 3.49 s to 3.89 s summed across eleven threads — **+0.40 s, or +29 µs per `examine()`** — about 37 ms of wall-equivalent. That is two orders of magnitude too small to explain a flat wall. The five extra `diverge()` calls are simply cheap next to the pixel loop they guard.

Splitting conserves total work; the regime is throughput-bound.

The real reason is in the same table: total compute (CPU+GPU pixel loops) is unchanged at ≈595–600 s (+0.9%, noise). Every pixel is iterated exactly once however the frame is partitioned, so subdividing a region *redistributes* work without reducing it. With eleven CPU workers and 100 frames in flight, no single region is on the critical path — while one thread spends 13 s on the frame-73 block, the other ten drain the remaining frames — so the wall is governed by total work over parallelism (≈595 s / 11 + GPU help ≈ 52 s), which 9-point leaves unchanged. The improved per-region distribution would only translate into wall time if a single region bound the critical path, which it does not at this thread count.

The GPU path is untouched.

Table 5 confirms the kernel total (45.5 s), the kernel max (137 ms), and the `cudaMemcpy` 99 %-kernel-wait signature are all unchanged; 9-point only shifts the GPU toward more, slightly smaller launches (avg 11.63 → 10.78 ms over 3,910 → 4,228 instances) with the same tail. The actual D→H transfer is ∼95 ms total — still not a bottleneck.

3.4. The binding cost is work, not balance

The decisive insight from the breakdown is that the heuristic can only move *where* work runs, never *how much* there is. The frame-73 block costs $960 \times 540 \times \approx 7{,}300$ iterations $\approx 3.8 \times 10^9$ iterations no matter how it is split; 9-point's finer decomposition cannot remove a single iteration. The load was already well balanced (per-thread spread under 2 % in both samplers), so improving balance further buys nothing. The only lever that can shrink the ∼595 s of total compute is one that *skips* iterating interior pixels — an interior certificate that constant-fills a region proven to be inside the set. This is why the wall is flat and why the next optimisation must target work, not subdivision.

3.5. Where 9-point *would* pay off

The flat result is configuration-specific. In a latency-bound setup — CPU-only, or a GPU-starved hybrid where a single 15 s region *is* the critical path — subdividing that region across threads would translate directly into wall time. The 9-point stencil's value here is therefore decomposition correctness (it makes the heuristic see interior structure) plus insurance for configurations where the long tail binds, not a throughput win at this thread count.

4. Recommendations

1. **Keep 9-point as the default sampler.** It strictly improves decomposition quality — one of two binding outliers eliminated, CPU per-region median -21% and max -14% (Table 6) — at a *measured* cost of +0.40 s CPU total (+29 µs/`examine`, ∼37 ms wall-equivalent; Table 4). It is the correct heuristic, and it is effectively free.

2. **Attack work, not balance — this is the real lever.** The breakdown shows total compute is conserved under any partitioning (Table 4); the only way to shrink the ∼595 s is to stop iterating interior pixels. Two options, in order of safety: (a) *periodicity checking* inside `diverge()` — detect the cycle that interior orbits settle into and bail early; exact, no image change, and it directly cuts the frame-73 block's 3.8×10^9 iterations; (b) a *heuristic interior certificate* that constant-fills a large region whose samples all reach `maxIter` — cheaper still but risks filling over a thin filament, so it trades a small accuracy risk for speed. The exact cardioid/bulb certificate (report 08) does not apply — frame 73 is a minibrot interior.

3. **Re-test 9-point in a latency-bound config.** Run CPU-only and GPU+1CPU; where a single region bounds the critical path, 9-point's tighter distribution (and the residual-outlier split of recommendation 2a) should convert into wall time that is invisible at 12 threads.

4. **Then the throughput levers of report 08 (async streams, priority queue)** — but note Table 5: the D→H transfer is only ∼95 ms, so async streams recover GPU-thread *idle* around the kernel, not transfer time.

5. Conclusion

The 9-point stencil does what report 08 predicted it would for the *less*-dense outlier: frame 89, 84 % interior, is detected and subdivided, dropping the count of >5 s regions from two to one and exposing +5.4% more leaves overall. It does *not* fix the densest region — frame 73, 97 % interior — whose thin divergent filament slips between all nine fixed sample points; sparse sampling is a probabilistic catch, not a guarantee. And at `diffT` = 0.1 on this 12-thread hybrid the change is wall-time-neutral (+0.6%, within noise). The Nsight Systems breakdown pins down why: the extra sampling costs a measured +0.40 s of CPU (∼37 ms wall-equivalent), far too little to matter, and splitting *conserves* total compute (≈595 s in both, +0.9%) — it moves work between threads but removes none. Since the load was already balanced to under

2 %, better decomposition has nothing left to gain at this thread count. The 9-point sampler is the right default on correctness grounds and is effectively free, but the wall is bound by total interior-pixel work, so the next optimisation must *reduce* that work — periodicity checking or an interior certificate (recommendation 2) — rather than subdivide it further.